An LM $p$ generates strings from $\Delta^* \subseteq (\Sigma \times Q)^*$:

$$\boldsymbol{x}_1 = q_0, y_1, q_1, q_2, \ldots, q_N, y_{T-1}, y_T \sim p$$

$$\boldsymbol{x}_2 = q'_0, q'_1, y_1, q_2, \ldots, y_T, q'_{M-1}, q'_M \sim p$$

The function $\phi$ removes the CoT steps:

$$\phi(\boldsymbol{x}_1) = \phi(\boldsymbol{x}_2) = y_1, y_2, \ldots, y_{T-1}, y_T \sim p^{💬}$$

Figure 2: An LM $p$ can generate strings from a state-augmented alphabet $\Delta$. The intermediate outputs $q_i$ allow it to condition on the previous states of the computation. By *post-hoc* removing the intermediate outputs with $\phi$, we obtain a CoT-augmented LM $p^{💬}$ over $\Sigma^*$ that generates more human-readable outputs.

exemplified in the cases of regular LMs, this can concretely increase the representational capacity of a model class, as in the case of Thm. 3.1. On the other hand, regular reducibility never *decreases* the representational capacity of a model class, since $\phi$ can always be set to the identity function. Due to the close connection between neural LMs and determinism (Svete and Cotterell, 2023), one could hope that a similar increase in capabilities could be achieved for neural LMs augmented with CoT reasoning as well. In §3, we show that this is indeed the case.

### 3.4 Regular Reducibility and CoT Reasoning

After introducing the notion of regular reducibility and presenting some of its core properties, we now use it to define CoT-augmented LMs.

**Definition 3.3.** *Given alphabets $\Sigma$ and $\Delta$ where $\Delta$ satisfies the $\Sigma$-augmentation condition, an LM $p^{💬}$ over $\Sigma^*$ is called a* ***CoT-augmented LM*** *induced by LM $p$ over $\Delta^*$ if $p$ is regularly reducible from $p^{💬}$. That is, there exists a regular function $\phi\colon \Delta^* \to \Sigma^*$ such that $p^{💬} \stackrel{\text{def}}{=} p \circ \phi^{-1}$.*

Def. 3.3 defines a CoT-augmented LM $p^{💬}$ through the LM $p$ that generates strings from $\Delta^*$ and then applies the regular function $\phi$ to the generated strings to obtain strings from $\Sigma^*$. This allows $p$ to output additional information while still defining a probability distribution over strings from $\Sigma^*$ (see Fig. 2). The weight of a string $\boldsymbol{y} \in \Sigma^*$ under $p^{💬}$ is then computed as the sum over the weights of all strings in the preimage of $\phi$, analogously to Eq. (6):

$$p^{💬}(\boldsymbol{y}) = \sum_{\boldsymbol{x} \in \phi^{-1}(\boldsymbol{y})} p(\boldsymbol{x}). \tag{11}$$

This is the approach we take in §4.2.2.

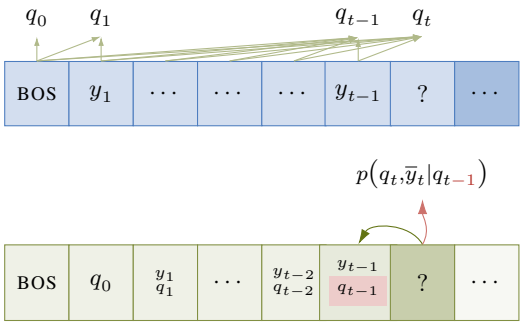


Figure 3: If $\boldsymbol{y}_{\leq t}$ uniquely determines a subpath in the PFSA, the current state $q_t$ can be uniquely determined *without* knowing the previous state (top). If, however, the substring $\boldsymbol{y}_{\leq t}$ can lead to multiple states, the relevant next-symbol distribution cannot be determined. Storing the *sampled* states as part of the output fixes this by keeping track of the sampled PFSA subpath (bottom).

## 4 CoT and Representational Capacity

We now connect the notions introduced in §3 to the representational capacity of neural LMs. We begin in §4.1 by showing how CoT reasoning endows autoregressive LMs with *non-determinism* by using scratch space to keep track of the current branch of execution. We then use similar principles to emulate probabilistic Turing machines with Elman RNNs (§4.2.1) and transformers (§4.2.2).

**Neural LMs and formal models of computation.** Probabilistic models of computation are often presented as autoregressive *generators* of strings that implicitly define probability (semi)measures (Icard, 2020). This interpretation is particularly important when addressing non-determinism, because it allows for multiple executions (for example, in the case of PFSAs, multiple different *paths*) generating the same string. Here, we treat neural LMs as autoregressive generators of strings and show that, by defining identical next-symbol distributions, they implicitly define the same semimeasures over strings as classical probabilistic models of computation.

### 4.1 Neural LMs and Regular LMs

We first discuss the connection between CoT-augmented neural LMs and PFSAs.

#### 4.1.1 Recurrent Neural LMs

Minsky's (1954) construction provided one of the first connections between a neural network and a formal computational model. It showed that RNNs can emulate (deterministic) FSAs, where the RNN accepts string by activating a particular

neuron after reading the string. The relationship in the probabilistic case was explored by Svete and Cotterell (2023), who show the equivalence of pure Elman RNNs (without CoT reasoning) and *deterministic* PFSAs.[15] This illustrates an important distinction between the deterministic and non-deterministic frameworks and thus a discrepancy between general PFSAs and RNN LMs. Intuitively, the discrepancy comes from the fact that there is no non-determinism in the recurrence of an RNN, which is thus unable to capture the possibly non-deterministic decisions of the PFSA. CoT reasoning endows an RNN with exactly this non-determinism because it allows the RNN to sample and refer back to *trajectories* rather than only symbols by annotating each of its outputs with the current state of the PFSA.[16] Upon reading the previously randomly generated state of the automaton captured in the output, it can follow the randomly sampled generating trajectories.

The following two theorems show that RNN LMs with fixed precision and CoT reasoning are weakly equivalent to general PFSA. This requires showing the correspondence in both directions, i.e., that *(1)* the distribution induced by any PFSA can be generated by a constant-precision RNN LM with CoT, and *(2)* any CoT-augmented constant-precision RNN LM can be emulated by a PFSA.

**Theorem 4.1.** *For any regular LM, there exists a weakly equivalent CoT-augmented constant-precision Elman RNN LM.*

*Proof intuition.* We show how, using the RNN's recurrence and output sampling step, we can implement the transition function of any PFSA. The RNN starts by sampling an output symbol containing an initial state $q_0$ according to the initial distribution $\lambda$ without emitting a language symbol. This output gets fed back into the RNN at the next time step, allowing it to *read* the sampled state, and the next symbol–state pair is sampled according to the conditional distribution defined by the output state, as illustrated by Fig. 3. Finally, the states in the generated string are removed by a regular function, leaving only the language output. See App. F for the detailed proof. ■

[15]The relationship between RNN LMs and *non*-deterministic PFSAs was explored by Svete et al. (2024), albeit with linearly bounded-precision RNNs.

[16]Recall that non-determinism means that multiple possible transitions to different states can yield the same symbol (§2.2).

**Theorem 4.2.** *For any constant-precision CoT-augmented Elman RNN LM, there exists a weakly equivalent PFSA.*

*Proof.* See App. F. ■

Thms. 4.1 and 4.2 establish the equivalence between CoT-augmented *constant-precision* Elman RNN LMs and general regular LMs. This illuminates the added representational capacity awarded by CoT reasoning: Storing the current FSA state in the output string and removing it later allows the model to handle non-determinism which is not possible without the additional information.

#### 4.1.2 Transformer LMs

We now show an analogous claim to Thm. 4.1 for Transformer LMs with CoT. This is simply a stepping stone towards full probabilistic Turing completeness—a similar construction will then lead us to the full proof that transformer LMs with unbounded precision can simulate PTMs.

**Theorem 4.3.** *For any regular LM, there exists a weakly equivalent CoT-augmented constant-precision transformer LM.*

*Proof sketch.* We use the construction from Svete and Cotterell (2024) which shows how to encode $n$-gram LMs in a transformer. Here, the alphabet of the transformer is augmented with the states of the PFSA, i.e., the symbol at each position $t$ contains not only an output symbol but also the current state of the PFSA at time $t$. Thereby each input symbol contains all the information required to compute the next-symbol probabilities (it is effectively a unigram LM). Because the next symbol probabilities in a PFSA only depend on the current state, the transformer does not need to use attention at all and can rely solely on the output of its residual connections. See App. F for the detailed proof. ■

### 4.2 Neural LMs and PTMs

We now extend the results from the previous section from representing the simple regular LM to expressing all enumerable semimeasures over strings by emulating probabilistic Turing machines. For this, in the RNN case, we require unbounded precision and $\mathrm{ReLU}$ activation functions rather than fixed precision and Heaviside activations.

#### 4.2.1 Recurrent Neural LMs

First, note the following definition:

**Definition 4.1.** *An autoregressive LM is called a* ***real-time LM*** *if it never outputs empty symbols ($\varepsilon$).*

Nowak et al. (2023) show that RNN LMs with rationally valued activations that are not restricted to operating in real time are Turing complete and weakly equivalent to a subset of rationally weighted PTMs. Intuitively, not operating in real time gives an LM additional computation time while storing the results of its computations in the hidden states. This is a form of CoT reasoning since we could equivalently let the LM output additional symbols that do not count toward the final output. [17] The ability to *erase* certain symbols, as done by our transducer $\phi$, helps make the setup of Nowak et al. (2023) realistic. Importantly, beyond giving the LM more computation time, CoT also allows the LM to model non-determinism in PTMs.

**Theorem 4.4.** *For every LM induced by a non-deterministic probabilistic Turing machine, there exists a weakly equivalent CoT-augmented RNN LM with unbounded precision.*

*Proof intuition.* The proof follows Nowak et al.'s (2023) probabilistic version of the proof by Siegelmann and Sontag (1992), but extended to account for CoT reasoning. For the ability to simulate *all* PTMs rather than just a subset, we augment the output alphabet of the RNN with enough information about the current PTM configuration so that subsequent steps can uniquely identify the previous action taken. Concretely, the output alphabet $\Delta$ contains information about the current state, the symbol written on the working tape of the PTM, and the head action performed. This additional information is then removed by our regular function at the end, yielding only the output of the simulated PTM generated according to its probability. A detailed proof is presented in App. F. ■

#### 4.2.2 Transformer LMs

The representational capacity of transformer LMs has received a lot of attention in the last few years. Pérez et al. (2021) established that the encoder–decoder variant of the architecture is Turing complete. Since then, concurrent work has shown non-probabilistic Turing completeness of LM-oriented decoder-only variants (Merrill and Sabharwal, 2024; Feng et al., 2023). We extend the work to the *probabilistic* case.

[17] On the other hand, some CoT-augmented LMs do operate in real time but over an extended alphabet; see §4.1.

**Theorem 4.5.** *For any PTM-induced LM, there exists a weakly equivalent unbounded-precision CoT-augmented Transformer LM.*

*Proof intuition.* The proof follows Pérez et al. (2021) but is adapted to the probabilistic case. Again, the main idea is to augment the output alphabet with enough information about the current PTM configuration to reconstruct the probabilities of possible actions at each time step. This information and appropriate positional encodings are enough to recover the PTM's current configuration and thus the next-action distribution, allowing us to construct a weakly equivalent transformer LM. A regular function then removes the additional information from the string. See App. F for details. ■

#### 4.2.3 Weak Equivalence

Thms. 4.4 and 4.5 show that transformer and RNN LMs with CoT reasoning are at least as powerful as PTMs. Weak equivalence requires us to also prove the reverse of these two theorems, analogous to Thm. 4.2 for constant-precision Elman RNN LMs.

**Theorem 4.6.** *For any rationally valued RNN LM and transformer LM with CoT reasoning, there exists a weakly equivalent PTM.*

*Proof.* RNN LMs define enumerable semimeasures (Nowak et al., 2023, App. G), which can always be expressed by a PTM (Icard, 2020, Thm. 3). Following the same reasoning, transformer LMs define enumerable semimeasures as well; the probability of a string $\boldsymbol{y}$ is defined as the sum probabilities of all runs of the transformer which result in the output $\boldsymbol{y}$, of which there are countably many. Finally, there is only a countable number of ways a regular function can transform any string, so RNN LMs and Transformer LMs with CoT still define enumerable semimeasures. ■

## 5 Conclusion

Many modern language models have been shown to perform better with CoT reasoning. Despite its empirical success, CoT reasoning has yet to be well understood formally. Recent theoretical work in this area has analyzed the Turing completeness of CoT-augmented LMs, failing to account for the inherently probabilistic nature of LMs. We fix this mismatch in the current literature by introducing a novel formalization of CoT reasoning.